Lab 4 - CSE 157

Lucas Reeves

June 1st, 2025

1 Introduction

This lab was all about bringing sensors, networking, databases, and a web application into one full-stack IoT system. The goal was to have our Raspberry Pis collect environmental data like temperature, humidity, wind speed, and soil moisture, and then push that data to a database hosted on one of our laptops. From there, we built a Flask-based web app that pulled the data from the database and displayed it in real time, along with forecast data from an online weather API. It's the first lab where we actually built something that could reasonably pass for a mini real-world system, which made it feel more complete compared to the more isolated tasks of earlier labs.

We reused a lot of the same components from Labs 2 and 3, including the sensors and the Pi-to-Pi communication setup. But what made this lab different was that we had to figure out how to get the Pis and the database to talk to each other over a network (two in our case... more on that soon) , and then wrap everything up in a working web app.

I'll go over the system design, how we set up the server and database, how we adapted the polling and token-ring configurations to work with a central database, and finally how we got the web app to visualize everything.

2 Overall Design

The design of this lab despite using most of the same components was significantly different than the other labs thus far. We of course used the Raspberry Pi's once again, as well as the same sensors connected directly to the board from lab 2 and lab 3. The new components in the mix are our actual laptops which will be running the database to store each of the Pi's sensor data. The database is packaged with XAMPP which also includes a web server so we can easily interact with the database from our browsers. Finally there is a separate web application made with the Flask web framework that displays the database data on a website accessible by others in the wifi network.

Before talking about the overall design, I have to clarify that our group did not fully understand the intended set up of the IoT system specified in the lab paper. The intended set up was to have each of the Pi's as well as the computer (or computer's) running the web server and data base all connected on the same infrastructure-based network in the lab. Our design is a hybrid network; the Pi's are all connected in their own ad-hoc network, while our database and web server connected to the lab's wifi network. We designated one Pi to not only collect data its own sensor data but also act as a gateway, switching between the ad-hoc network to compile the other Pi's data and the wifi network in order to send the information to the database.

Despite this being different to the intended network configuration, our system still operates almost entirely the same though significantly slower since we have the added latency of having to wait for one of the Pi's to switch from ad-hoc to managed mode and back again. This got us some confused looks from Andrea at first but we are very thankful that she still determined the overall functionality of our lab was still adequate!

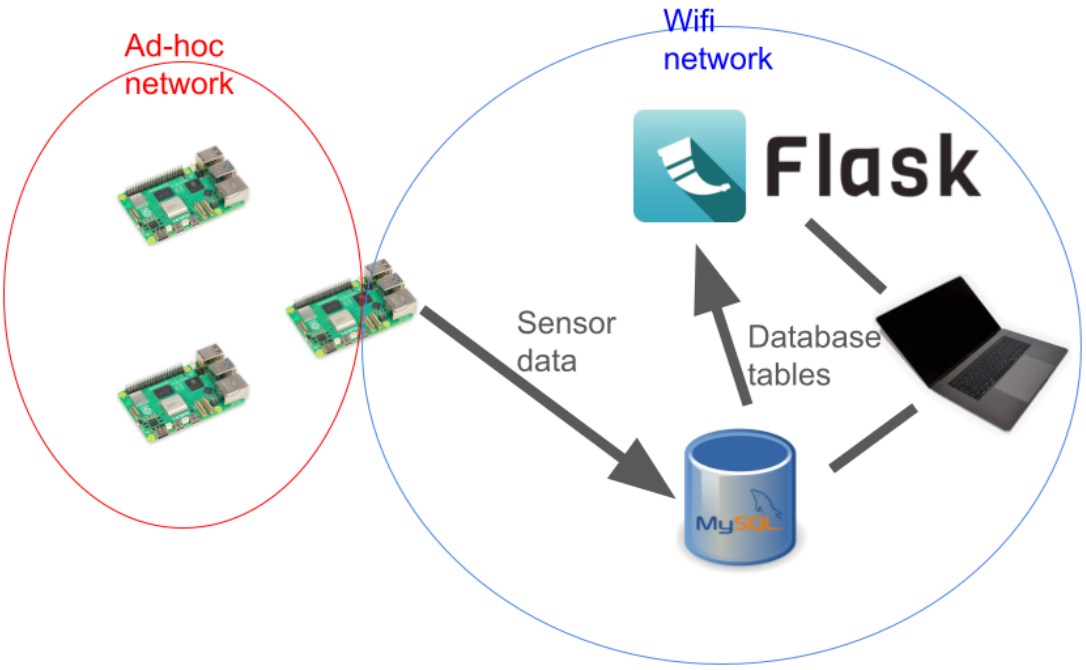

Figure 1: A Diagram of the IoT System

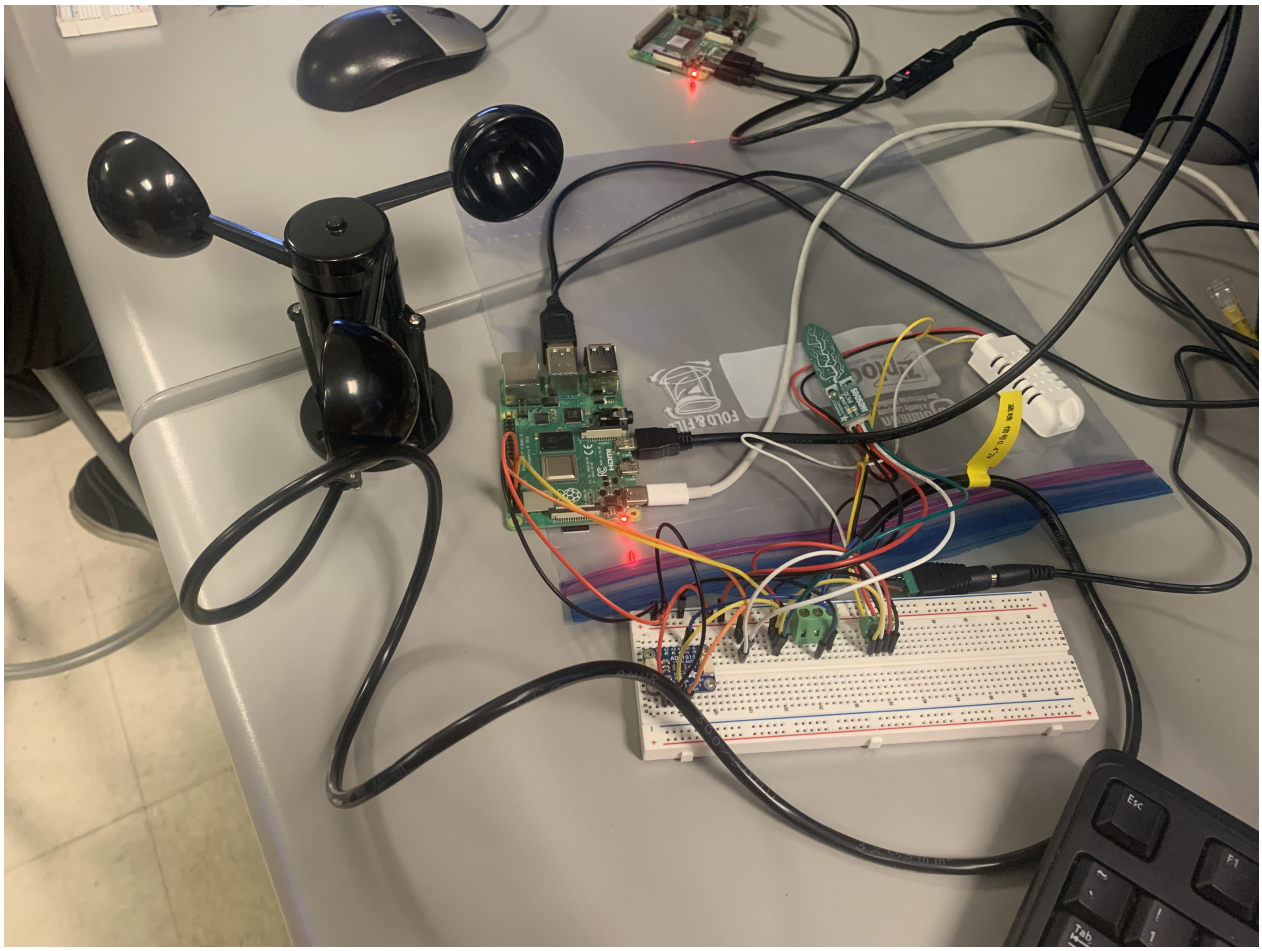

Figure 2: A picture of the Pi setup with sensors

3 The Web Server

The apache web server included in XAMPP was the most hassle free tool we used in this lab. As soon as I had XAMPP installed and running on my computer, the application told me the apache server was running and I was able to access its web page by visiting localhost on my browser. I found the web server to be really handy, especially since I had never worked with a mySQL database before and could avoid having to learn some inconsequential mySQL commands to do things like clear one of the tables in the database or create a user with certain permissions by using the included phpMyAdmin web interface. The only confusion I had with this part of the lab was really with the XAMPP interface it self. In the tutorial linked on the lab document, it seems their examples use an older version of windows with a different interface than the application on my Macbook, but after poking around a bi this was never an issue. Overall, not too bad on the web server side of things with XAMPP.

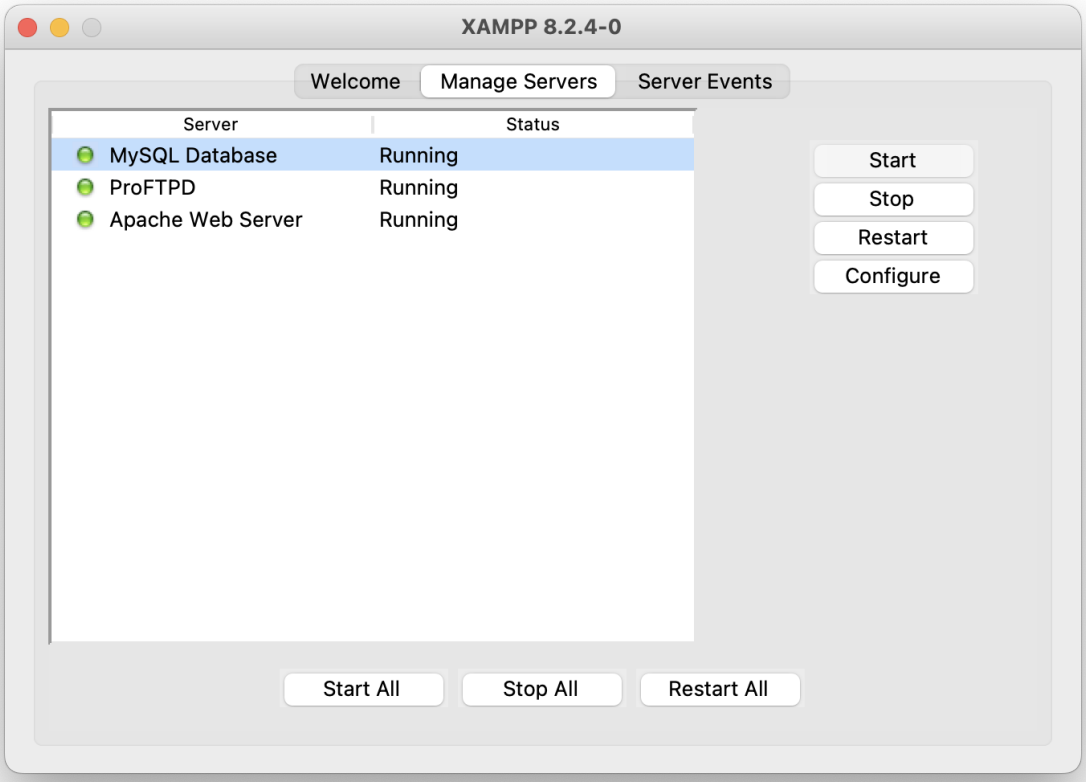

Figure 3: The main page for the macOS XAMPP application

4 The Database

The MySQL database included in the XAMPP bundle definitely gave me some more headache than the web server. The first time I launched the mySQL database I thought everything was good to go. XAMPP told me it was running, and I assumed that I could just connect to it through my Macbooks IP address with python connector after setting up user credentials through phpMyAdmin. This worked for my group mates on their windows machine but for whatever reason I just could not connect to the database. I looked around online to see if I had to set a special address for the database on my computer or something and was led to a setting in mySQL's config file, **bind-address**. This settings tells the mySQL server what network interface to listen to, and by default it was set to 127.0.0.1 to listen to local network interfaces. I ended up being able to connect to the database on the pi's after I set this address to the address of my mac on the network. My guess as to why this worked is that the default bind address made the mySQL server only accessible by the

system running it, but at the same time my group mates did not need to do this step so I am still unsure.

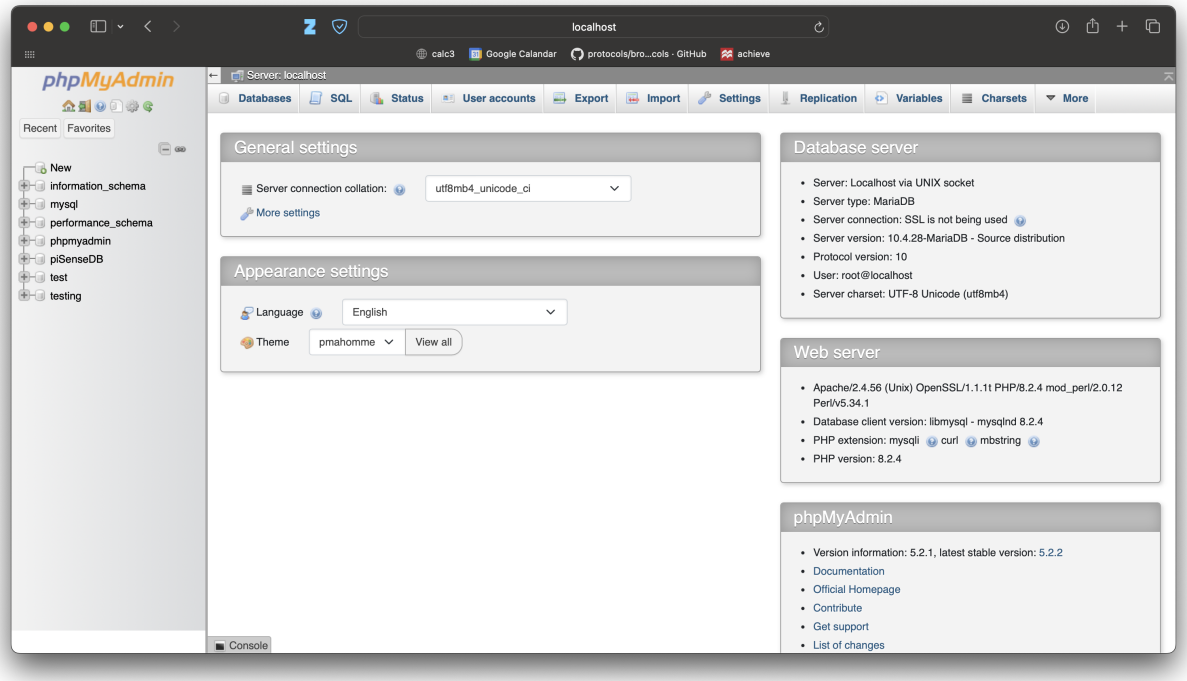

Figure 4: phpMyAdmin page on the Apache web server

I ran into another emergent problem with this where after I switched networks on my machine such as from my home wifi network to the labs wifi network and therefore had to change this bind address, I was not able to restart the mySQL server. I would press the restart button on the mySQL server through the XAMPP application and it would log that the database was starting and then just stop with no further log messages. This was fixed by restarting my computer every time I had to change the bind-address, which was pretty annoying.

Aside from the bothersome set up, it was actually easy to configure the database once I got it working. We set it up identical to the lab instructions, with the PiSenseDB database and then a table for each of the Pi's sensor data, three tables in total. Each of the tables had float type entries for temperature, humidity, wind speed, and soil moisture, and then a timestamp type entry for timestamps.

To create the database, tables, and user for the Pi's I used the phpMyAdmin interface provided on the Apache web server. For pushing data to the database on the Pi's I used the mySQL command **INSERT INTO (tablename)** command, and for retrieving data from each of the tables I used the **SELECT (entryname) FROM (tablename)** commands for the web application. While this was the most annoying part of the lab, this was the first time I worked with a database and it was cool seeing a little bit of how they

work and I can for sure see some very useful real world applications for them.

Schema of sensor_readings Tables in piSenseDB

Column Name	Data Type
timestamp	TIMESTAMP
temperature	FLOAT
humidity	FLOAT
wind_speed	FLOAT
soil_moisture	FLOAT

Figure 5: Simple diagram of the database schema

5 Adapting the Configurations

Between the two approaches we used - polling and token ring - each has its strengths and quirks depending on the situation. The polling-based configuration is pretty rigid: the primary Pi initiates all communication and always asks each secondary Pi for its data, even if that Pi doesn't have anything to report. This makes the system very predictable and reliable in cases where devices might be flaky, but it also introduces unnecessary communication when some Pis have no new data. On the flip side, the token ring approach is more hands-off and self-managed. Each Pi just passes the token to the next one, and only sends data if it has some to send. This makes it more efficient in cases where not every Pi has something to report, but introduces complexity when handling failures or when everything is generating data at once.

We had to think a bit about robustness, especially given how often things went awry in lab 3. For the polling system, robustness was mostly about making sure the primary Pi didn't hang forever waiting for a secondary Pi that crashed and could adapt and run by itself. We added a few timeouts and had the primary pi remove any any Pi from its polling list that didn't respond after a few seconds. It's a little hacky, but it kept the system running. Token ring was a lot more complicated. The problem there was if a Pi failed while holding the token, the system would basically freeze. Our solution was to have a timeout on receiving the token - if a Pi didn't get the token again in a certain window, it assumed something went wrong and tried to reinitialize the ring using whatever Pis were still responding. This wasn't super elegant, and actually created a duplicate token if a pi left the ring with the token and joined again but in practice it got the job done and kept things rolling.

We also considered the case where the Pi responsible for connecting to the database dies. In the polling approach, that's always the primary, so if it goes down, the whole system has to be restarted. In the token ring configuration, any Pi can act as the connector. If the current connector fails, the ring eventually designates

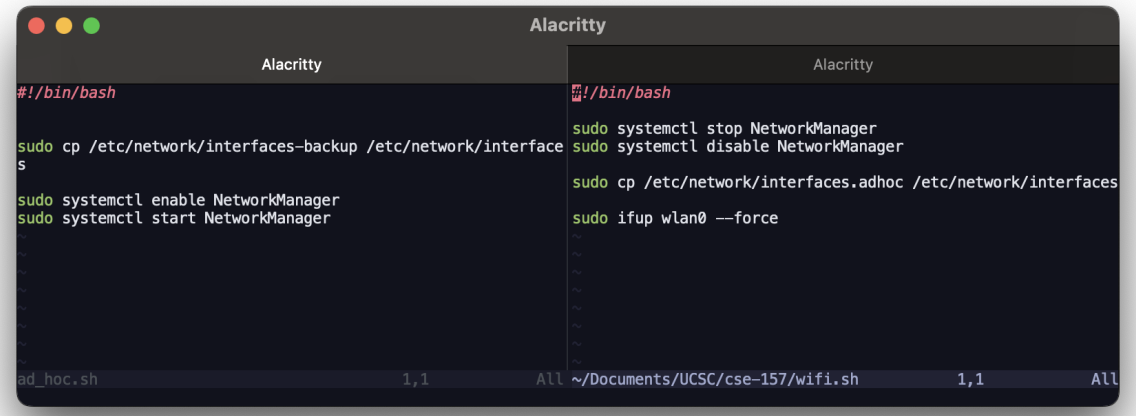

Figure 6: The two scripts embedded in both configurations to switch between ad-hoc and wifi

another Pi to take over, assuming the re initialization logic kicks in correctly. This gave token ring a bit more resilience in that respect, even if it was a bit slower to recover.

In terms of fairness, the polling system obviously isn't fair. The primary Pi controls everything, and the secondaries only get to speak when spoken to. That might be fine for basic data collection, but it's not ideal if the secondaries have lots of data or need equal bandwidth. Token ring, on the other hand, is a lot more fair. Every Pi gets a turn to send, regardless of who it is, and it naturally balances the load over time. That said, if one Pi is slower, it can still drag down the whole loop.

We were also asked to consider other topologies mesh or star networks. Mesh would've been cool but was probably too ambitious for this lab, especially with how flaky the Pis were at times. A star topology might have worked better if we had a dedicated router or access point (like what was initially intended for this lab). It wasn't ideal, and definitely added some latency, but it got us to a working system without totally blowing up the lab setup.

6 The Web App

For the web app, we used Flask to serve a simple website that visualizes the data stored in our MySQL database from each of the Pis. The main job of this app was to pull all the sensor data for temperature, humidity, wind speed, and soil moisture from the three tables in the `piSenseDB` database, and then show that data in a set of graphs along with the forecasted weather data. We used the Python MySQL connector to talk to the database, and the `requests` library to pull forecast data from the OpenMeteo API (same way

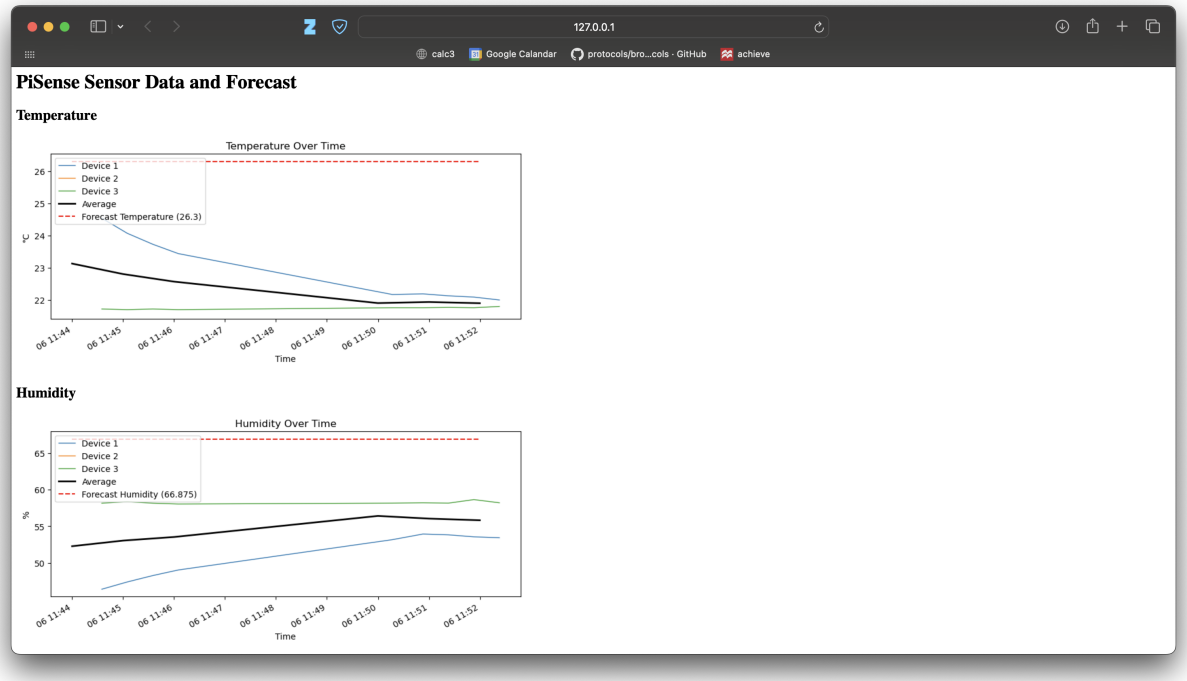

Figure 7: An aggressively mediocre web app (half the page)

we did it back in Lab 1).

The actual logic in `web-app.py` wasn't too complicated. Each time the root route was hit, the app would connect to the database, run SELECT queries on each table to get all the sensor readings, and then package everything into arrays of timestamps and values. I computed the average reading across all three Pis for each sensor type, and then plotted everything using Matplotlib. For the forecast, I just pulled the current weather prediction once per reload and then added it to each graph as a flat reference line so you can easily compare what the sensors were reading to what was expected.

I didn't bother with any front-end tools besides raw HTML and the graphs themselves, since Flask makes it easy to serve Matplotlib plots as images. The site lacks pizza but it works. There's a separate graph for each measurement type, and each one shows the three Pis' data in different colors, along with the average and the forecast (if available). The timestamps are on the x-axis and everything is spaced out nicely enough that you can see how the values changed over time.

To make the app visible to others on the network, I ran the Flask app with `host='0.0.0.0'` and made sure my firewall wasn't blocking the port. After that, people could just type in my IP address and see the data live in their browsers.

One design limitation I ran into was with the creation of the plots. When I first wrote the program the web page was really slow to load the graphs. Usually it would take a couple refreshes of the page to actually

get all four of the graphs to appear. I fixed this by simply setting the **threaded** flag for the Flask web server to true, which sped up the loading significantly.

7 Conclusion

This lab was the most complete system we've built so far and also the most frustrating at times. There were a lot of moving pieces between the networking, database, and web app, and getting them all to play nice took more time than I expected. But by the end, it was really satisfying to see live sensor data show up in a web browser, knowing it had made its way from the Pis, through the network, into the database, and finally into a graph on a Flask site.

One of the biggest takeaways for me was how finicky real-world networking can be, especially when you're bouncing between ad hoc networks and WiFi. I got a good crash course in how databases work, both in terms of structure and permissions, and how even something small like a config file setting can block everything from working. Finally, actually building a web app (even a super basic one) that displays live data made the project feel much more grounded and practical. It was a pain at times, but I definitely learned a lot, and it was cool to finally connect all the dots from the past few labs into something that felt like a real application.